

# Numbered elements and the vision option

Week 2, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026

## What we will cover

- The indexed selector map: how integers get assigned
- Element index as the agent's vocabulary
- Why this replaces CSS selectors and XPath
- Optional screenshots and `use_vision`
- DOM-first vs. pure-screenshot: token cost and tradeoffs

# Part 1: the selector map

## From filtered set to numbered list

- After visibility filtering: a set of interactive, visible elements
- The DOM service assigns sequential integers: 0, 1, 2, 3, ...
- Each integer maps to an enhanced node object in the `selector_map`
- The node object carries: role, name, type, bounding box, backend node ID
- The backend node ID is what gets sent to CDP when the agent acts

## What the agent receives

The LLM sees a text representation like:

```
[0] link "Home"      href=/
[1] link "About"     href=/about
[2] link "Sign in"   href=/login
[3] button "Search"
[4] input  type=text  placeholder="Search..."
[5] button "Submit"   type=submit
```

This is the entire “page” from the agent’s point of view.

## The index is rebuilt every step

- After every navigation or page change, the full extraction runs again
- Indices from the previous step are invalid; new indices are assigned fresh
- This prevents stale references: the agent always acts on the current page state
- Cached indices would break silently when dynamic JS adds or removes elements

## Part 2: the index as the agent's vocabulary

## Why integers, not selectors?

- CSS selectors break when class names change (and they do, on every deploy)
- XPath expressions are brittle to layout changes
- Pixel coordinates drift when viewport size changes
- Integers reference the DOM service's own lookup: stable for this step, correct by construction



## The agent's decision in practice

The LLM receives the numbered list, reasons over the task, and produces:

```
{  
  "next_goal": "Fill in the search box and submit",  
  "action": [  
    { "input_text": { "index": 4, "text": "browser automation" } },  
    { "click_element": { "index": 5 } }  
  ]  
}
```

No selector. No XPath. Two integers.

## Bounded action space

- The agent can only reference indices that exist in the current selector\_map
- Referencing a nonexistent index produces an `ActionResult` error, not silent failure
- This is a correctness property: every element-targeted action points to a real, visible, interactive element
- The framework enforces what the LLM cannot always enforce itself

# Part 3: the vision option

## `use_vision`: adding a screenshot

- Default: DOM-first, no screenshot
- `Agent(task=..., llm=..., use_vision=True)` sends a low-detail screenshot with each step
- The screenshot accompanies the numbered text description; it does not replace it
- The agent can align the numbered overlay on the screenshot with the text entries

## When vision helps

- Multi-column forms where label and input are not adjacent in the DOM
- Pages where layout position carries meaning (sidebar vs. main content)
- Confirming that an action produced the expected visual result
- Tasks where the agent needs to describe what it sees ("read the chart")

## When vision adds cost without benefit

- Pure navigational tasks: clicking a clearly labeled link
- Standard form filling: filling in a username and password
- Tasks that are fully described by the element roles and names
- Any step where the text description is unambiguous

# Part 4: DOM-first vs. pure screenshot

# Three architectures (Sokolov, 2025)

| Architecture       | What the agent receives                | Token cost per step            |
|--------------------|----------------------------------------|--------------------------------|
| Screenshot only    | One image of the page                  | High (1,000+ tokens for image) |
| Accessibility tree | Structured text: roles and names       | Low (500 tokens typical)       |
| Runtime DOM + a11y | Text description + optional screenshot | Low, optionally medium         |

browser-use occupies the third column: DOM plus accessibility tree, screenshot optional.



## The token arithmetic

- Low-detail screenshot in OpenAI API: 85 tokens minimum
- High-detail screenshot of a large page: 1,000+ tokens for the image alone
- 20-step task, DOM-only vs. DOM + screenshot: thousands of extra tokens
- For a text-only model, screenshots are unavailable regardless of cost
- For a vision model on a simple navigational task, screenshots are wasted cost

Gonsalvez (2025): the DOM-first approach is “fast and token-efficient” compared to screenshot-only alternatives.

## What pure screenshot cannot handle

- Requires a vision-capable LLM (rules out cheaper text models)
- Acts on pixel coordinates that drift with viewport changes
- Each screenshot is a snapshot; dynamic content requires re-screenshotting
- No structured element list: the model must infer interactivity from pixels

## What DOM-first cannot handle

- Canvas-based rendering (interactive charts, games, map UIs): no DOM content to extract
- Pages with poor accessibility compliance: labels missing, click handlers undetectable
- Tasks that require visual reasoning ("which button is highlighted?")

These are the cases where `use_vision=True` is the right choice.



Instead of taking a screenshot and feeding pixels to a vision model, browser-use extracts the DOM structure and presents it as text the model can reason over.

## Take-aways

1. The `selector_map` assigns a sequential integer to each surviving interactive, visible element. Integers are the agent's only vocabulary for addressing the page.
2. The index is rebuilt from scratch at every step: it always reflects the current page state.
3. `use_vision=True` adds an optional low-detail screenshot for spatial context, at a measurable per-step token cost.
4. DOM-first works with text-only models and costs fewer tokens. Pure-screenshot works on any UI but requires vision capability and is more expensive.
5. Choose `use_vision=True` when layout or visual confirmation matters; leave it off for standard navigational and form-filling tasks.

## What's next

- **Section (this week):** Dump the selector map for three real pages and match indices to on-screen elements
- **Week 3 reading:** The browser-state summary and how the DOM output is assembled into the full perceive payload
- **Week 3 lecture:** Configuring a `BrowserSession` and inspecting the state it captures